Computational Vision

Laboratory Guide

Faculty: Prof. Alexandra Moutinho
Eng. Alexandre Athayde

February 2026

Introduction

The present document has the objective to aid the students in the laboratory component of the curricular unit **Computational Vision**, describing the objectives and required tasks to develop at each laboratory session, which aim to apply key-concepts and algorithms of Computer Vision (CV).
Following CV's syllabus, this guide is organized into two different parts, each of them approaching separate topics of this course. The first one, *Image Processing*, presents some fundamental ideas about computational vision using a single camera and the common techniques for processing of images. This first part is then complemented by providing comprehensive insight about feature detection after the processing stage, exploring multiple techniques. In the second part, *Computer Vision*, an introduction of using deep learning for feature recognition is made, and the fundamentals of projective geometry and camera models are introduced. These are used to explore the topic of stereo vision, which is particularly useful in distance measuring and 3D reconstruction.
Each laboratory session will provide certain skills that are useful in understanding the theoretical basis of CV, and, furthermore, these are expected to help in the development of the CV projects.

- Reading materials:
  The laboratories require some level of preparation, and therefore reading materials are provided to the students using the **CV partners webpage**, and will be updated as the classes progress. These materials are the **Theoretical Classes slides** and the respective sections of the **bibliographic references**, and, in specific classes, will be complemented with other materials. Particularly for the laboratory sessions, two courses offered by MathWorks will be helpful in Part 1, **Introduction to Image Processing** (link) and **Image Segmentation, Filtering, and Region Analysis** (link), and can be audited for free from the Coursera platform.
  Since the implementation will be made using MATLAB, a set of recommended MATLAB functions (denoted in a specific font: `function`) and/or Simulink blocks (represented in italic: *Simulink Block*) is provided in the guide for each laboratory session.

- Evaluation:
  At the end of each laboratory session, a quiz will take place, with a small number of questions to be answered directly about the topics addressed in the session. These questions can be relative to the respective lab and about theoretical concepts, so adequate preparation is advised. Note that, during the implementation during the laboratory sessions, you will work in groups of 3 students, but the quiz at the end of each session is individual.

Final Note: the recommended version of MATLAB is **2023a**, with the respective Simulink version, so please be sure to install it. If you are unable to do so, you may use the computers available in the laboratory. The required toolboxes for MATLAB are the ***Image Processing Toolbox***, ***Computer Vision Toolbox***, ***DSP System Toolbox***, ***Statistics and Machine Learning Toolbox***, and the ***Deep Learning Toolbox***, and, if cameras are to be used, the necessary drivers for the operational system and the ***Image Acquisition Toolbox Support Package for OS Generic Video Interface*** add-on should be installed as well.

Contents

I Image Processing **2**

1 Colour Spaces and Histograms **3**

Part I

Image Processing

Chapter 1

Colour Spaces and Histograms

Following the first-week lectures, the respective laboratory session addresses the representation of digital images, namely different colour spaces and how these are related, and provides some initial insight about the histogram of grayscale images.

Objectives

- Familiarization with image loading and visualization tools in MATLAB and Simulink
- Experiment different colour spaces
- Learn about the histogram of a grayscale image

MATLAB and Simulink functions

- Image loading: `imread`, *From Video Device*, *Image From File*
- Image visualization: `imshow`, `montage`, *To Video Display*
- Colour space conversion: `rgb2gray`, `hsv2gray`, `rgb2hsv`, `im2gray`, *Colour Space Conversion*
- Histogram visualization: `imhist`, *2-D Histogram and Array Plot*

Tasks

1. Load and visualize an image in MATLAB or Simulink

Expected time of completion for task: 10 minutes.

Naturally, the first step in image processing is to load an image. The image to be loaded should be a coloured one since it will be useful to explore different colour spaces. Do not forget that the image file should be in the working directory of MATLAB, or the path to it should be added.
The image used as an example is provided in the file *bottlecaps.jpg*, and is the following:

Figure 1.1: RGB example image.

2. Experiment using different colour spaces

Expected time of completion for task: 20 minutes.

After this, try converting this image into different colour spaces, namely HSV, YCbCr and grayscale. If you are using MATLAB, the `montage` command is useful to visualize multiple images. Alternatively, in Simulink, different *To Video Display* blocks can be used. The result should be similar to the following:

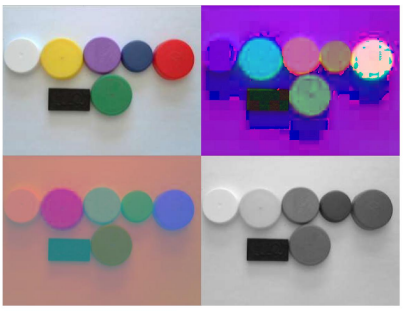

Figure 1.2: Example image in RGB (top left), HSV (top right), YCbCr (bottom left) and grayscale (bottom right).

Do not forget: you are not visualizing the images in different colour spaces, as `imshow`/`montage` interprets them in RGB; you are seeing the interpretation of RGB of a different colour space!

3. Obtain and visualize the histogram of a grayscale image

Expected time of completion for task: 10 minutes.

Lastly, obtain the histogram of a grayscale image and visualize it. As an example, a different image can be used, namely the one provided in the file *autumn.tif*, which is built-in in MATLAB (and therefore you do not need the file itself). Both the original coloured image and the respective grayscale conversion are shown on figure 1.3:

Figure 1.3: RGB and grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

The respective histogram is:

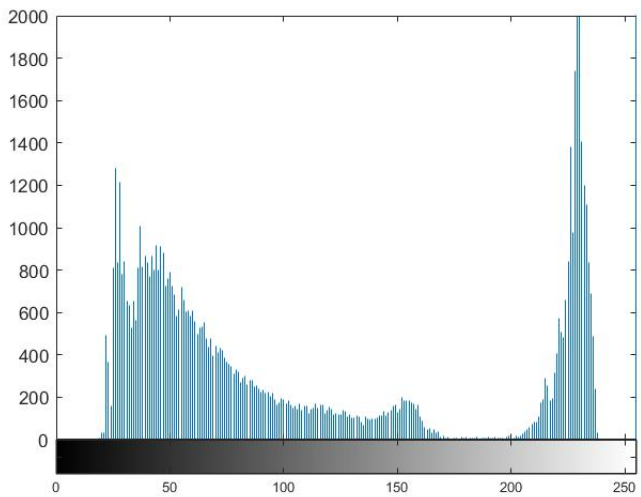

Figure 1.4: Histogram of the grayscale-conversion of the *autumn.tif* image (credited to MathWorks).

Experiment obtaining the histograms of different images and try to draw conclusions about how each of the histograms is related with the original grayscale image. What is expected to be the histogram of a dark image? Or a bright one?

Further work: correctly select a certain colour

Knowing how the different colour spaces work, it is now possible to filter the pixels of an image based on the values they present on a given colour space. For an 8-bit image in the RGB colour space, it can be assumed that a pixel is white if all its three components are above a certain value, which can be, for example, 200. Afterwards, it is possible to design an algorithm that generates a new image from the first one, separating the pixels that fulfil the mentioned condition and the ones that do not. Such procedure is presented on Algorithm 1, and the results shown in figure 1.5. Despite being somewhat more intricate, a similar algorithm can be used in Simulink. Please note that the separation is obviously not perfect, but this topic will be addressed in detail in further theoretical and laboratory classes.

Algorithm 1 Detecting specific colours in an RGB image

```
[a, b, c] = size(RGB)
                                        ▷ Extracts size of the original RGB image.
NewImg = zeros(a, b)
    ▷ Generates a matrix with zeros with the same number of rows and columns of the RGB image.
for i = 1, ..., a do
    for j = 1, ..., b do
        if RGB(i, j, 1) > 200 and RGB(i, j, 2) > 200 and RGB(i, j, 3) > 200 then NewImg(i, j) = 255
                                        ▷ If all components of the pixel are above 200, it assumes it as white.

        else NewImg(i, j) = 0
                                        ▷ If not, it assumes the pixel as black.
        end if
    end for
end for
```

Figure 1.5: Detecting white-coloured pixels in figure 1.1.

Try to use the procedure described above to detect each of the colours of the bottle caps in figure 1.1. Think about which colour space is more adequate, and what conditions should be imposed in the values of the pixels. Please note that the goal here is to provide a general understanding of the colour spaces and how these can be used to extract information from images, and not to detect all the bottle caps without flaw.